

CUADERNO DE PRÁCTICAS // DESARROLLO WEB

UD1 • UD2 SPRING BOOT

De un proyecto vacío a una API REST

Kerin Aguilera

JAVA 21 • SPRING BOOT • OCTUBRE 2026

✦ TUTORIA UNO

// Inicio

Aqui empezamos a traducir lo que es a traducir y replicar las practicas de las tutorias, al comenzar creamos el proyecto como simepre, esta bes teniendo la dependecia d Thymeleaf. Tras exportar ahora tendremos que hacer lo siguiente, creamos un controller prematuro.

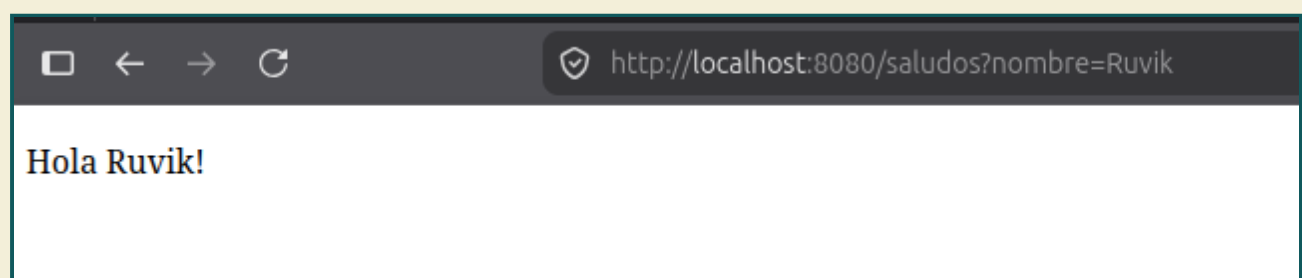
```
package ruvik.demoformulario;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class ControllerDemoFormulario {
    @GetMapping("/saludos")
    public String saludos(@RequestParam(name = "nombre",required = false,defaultValue = '
    Model modelo) {
        modelo.addAttribute("nombre", nombre);
        return "saludos";
    }
}
```

La idea de este ejercicio es pode pasarle por URL un nombre y este que saludo al nombre o por default si no se le pasa nada, que salude al mundo. Como indica lo que es el html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>SALUDOS</title>
</head>
<body>
    <p th:text="|Hola ${nombre}!!"></p>
</body>
</html>
```



✦ TUTORIAL 2

// Inicio

ahora damos un salto y aprendemos algo nuevo y es que ahora entra en juego lo que es las Entidades o los pojos donde guardamos datos directamente en un objeto

```
package ruvik.demoformulario;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ControllerDemoFormulario {
    @GetMapping("/saludos")
    public String formularioSaludos(Model modelo) {
        modelo.addAttribute("saludo", new Saludo());
        return "saludos";
    }

    @PostMapping("/saludos")
    public String saludosEnviado(@ModelAttribute Saludo saludo, Model modelo) {
        modelo.addAttribute("saludo", saludo);
        return "resultado";
    }
}
```

Existen conceptos nuevos, distintos. Tenemos el `new Saludo()` inicializamos el objeto y cuando hagamos el formulario guardamos esos datos.

otro concepto nuevo es el `@ModelAttribute` donde intuyo que decimos que queremos recibir un objeto, ese mismo objeto ser servira en la pagina donde visualizamos los datos. Pero vayamos por parte, miremos el formulario

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>SALUDOS</title>
</head>
<body>
  <h1>Formulario</h1>
  <form action="#" th:action="@{/saludos}" th:object="${saludo}" method="post">
    <p>Id: <input type="text" th:field="*{id}" /> </p>
    <p>Mensaje: <input type="text" th:field="*{contenido}" /> </p>
    <p>Id: <input type="submit" value="submit" /> <input type="reset" value="reset" />
  </form>
</body>
</html>

```

aqui tambien existen varios conceptos distinto a los que ya conocemos

`th:action="@{/saludos}"` este sirve para decirle al formulario que cuando se le de al botón de enviar, vaya directamente a ese sitio web `th:object="${saludo}"` Este concepto segun lo veo, le dice el formulario que los campos que tiene que rellenar son de este "objeto" que se llama saludo

`th:field="*{id}"` este funciona para decirle al formulario que este input, este lugar donde introducimos datos van otra relacionada los datos del siguiente objeto,

importante a parte de que nombre es distinto este no tiene el simbolo de `$` sino `*` es una diferencia importante y que hay que recordar.

Y ahora el resultado es lo siguiente:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Saludado</title>
</head>
<body>
  <h1>Resultado</h1>
  <p th:text="|id: ${saludo.id}|"></p>
  <p th:text="|contenido: ${saludo.contenido}|"></p>
  <a href="/saludos">Envía otro saludo</a>
</body>
</html>

```

Aqui tenemos nuevas cosas, aunque seguimos mostrando los datos como siempre aqui tenemos `th:text="|id: ${saludo.id}|"` que aunque es parecido, aqui le decimos que parte de ese objeto queremos mostrar y ver.

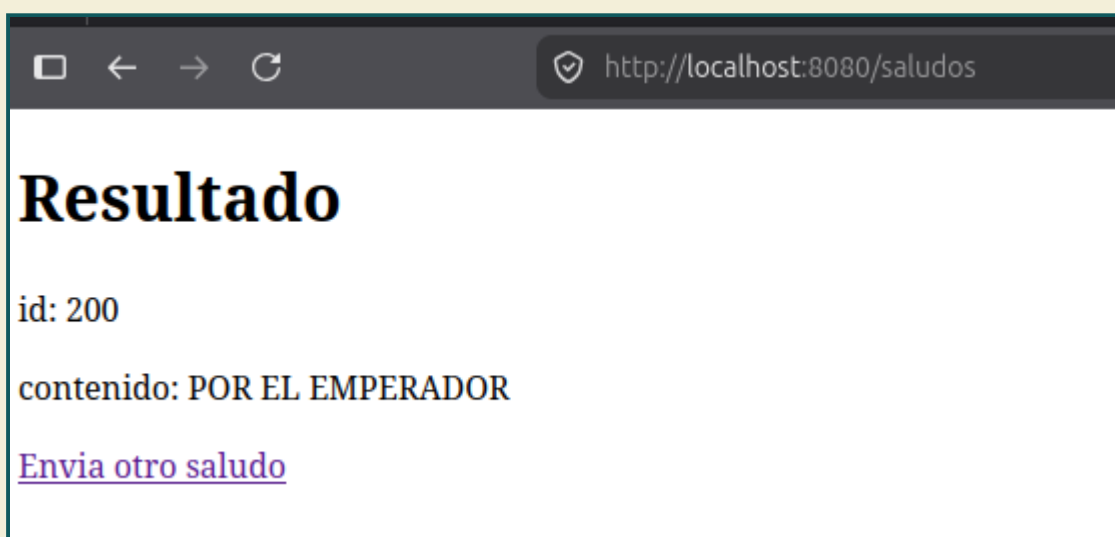
introducimos datos en el formulario

Formulario

Id:

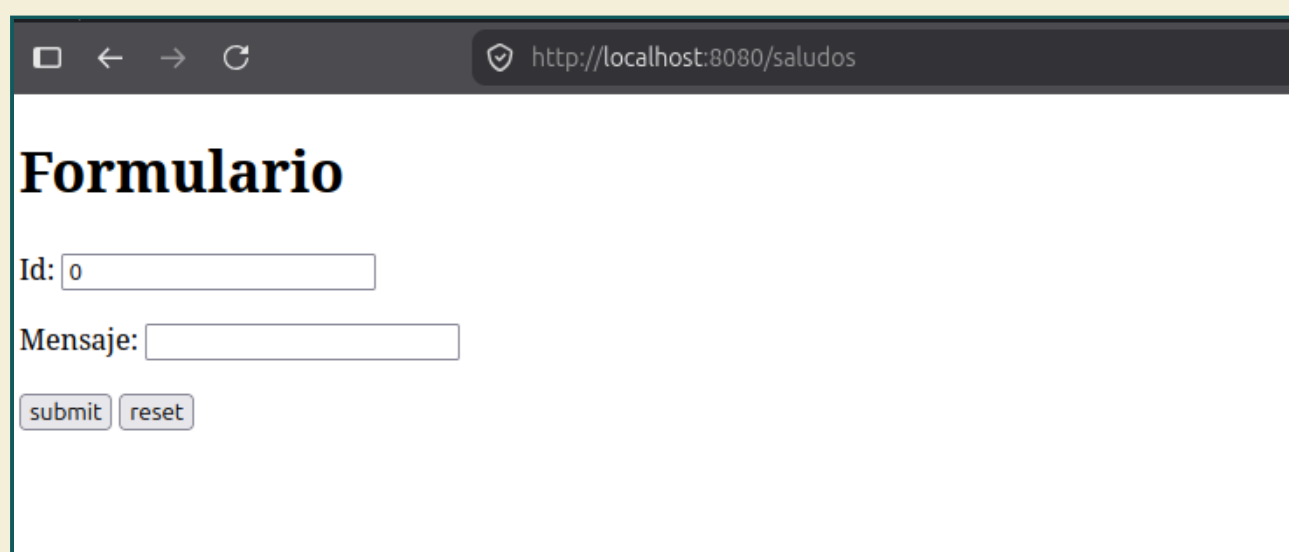
Mensaje:

Luego vemos el resultado:



The screenshot shows a web browser window with the URL `http://localhost:8080/saludos`. The page has a title **Resultado**. Below the title, it displays the submitted data: `id: 200` and `contenido: POR EL EMPERADOR`. At the bottom, there is a purple underlined link that says [Envia otro saludo](#).

y por ultimo podemos volver atras para hacer nuevos datos



The screenshot shows the same web browser window with the URL `http://localhost:8080/saludos`. The page has a title **Formulario**. Below the title, there are two input fields: `Id:` and `Mensaje:` . At the bottom, there are two buttons: `submit` and `reset`.

✦ TUTORIAL 3

// Inicio

imperial Aqui empezamos con otra cosa nueva, antes vimos los objetos aqui veremos las validaciones. aqui hay muchas cosas de las cuales hablar primero creamos lo que sabemos un objeto

```
package ruvik.validacionformulario;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class FormularioPersona {
    @NotNull
    @Size(min=2, max = 30)
    private String nombre;

    @NotNull
    @Min(18)
    private Integer edad;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Integer getEdad() {
        return edad;
    }

    public void setEdad(Integer edad) {
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "FormularioPersona{" +
            "nombre='" + nombre + '\'' +
            ", edad=" + edad +
            '}';
    }
}
```

luego aqui es donde todo cambia, hay mas cosas nuevas

```

package ruvik.validacionformulario;

import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Controller
public class ControladorWeb implements WebMvcConfigurer {

    public void addVistaControlador(ViewControllerRegistry registro) {
        registro.addViewController("/resultado").setViewName("resultado");
    }

    @GetMapping("/")
    public String verFormulario(FormularioPersona persona) {
        return "formulario";
    }

    @PostMapping("/")
    public String verificarInformacionPersona(@Valid FormularioPersona persona, BindingResult bindingResult) {

        if (bindingResult.hasErrors()) {
            return "formulario";
        }
        return "redirect:/resultado";
    }
}

```

Segun he buscado

```

public void addVistaControlador(ViewControllerRegistry registro) {
    registro.addViewController("/resultado").setViewName("resultado");
}

```

Esto funciona para mapear url directamente en java de html o mas es mapear url directamente en la plantilla o la vista.

Y ahora tenemos

```

public String verificarInformacionPersona(@Valid FormularioPersona persona, BindingResult bindingResult) {

```

Lo que hace `@valid` es decir que este formulario tiene validaciones y `BindingResult` es quien arroja el resultado de las iteraciones del usuario.

me han salido muchos errores investigando con la IA concretamente con Gemini me surgió lo siguiente porque no me funcionaban las validaciones

```

package ruvik.validacionformulario;

import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Controller
public class ControladorWeb implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registro) {
        registro.addViewController("/resultado").setViewName("resultado");
    }

    @GetMapping("/")
    public String verFormulario(@ModelAttribute("persona") FormularioPersona persona) {
        return "formulario";
    }

    @PostMapping("/")
    public String verificarInformacionPersona(@Valid @ModelAttribute("persona") FormularioPersona persona, BindingResult bindingResult) {
        if (bindingResult.hasErrors()) {
            return "formulario";
        }
        return "redirect:/resultado";
    }
}

```

Luego fallos pequeños como no cambiar los nombre del ingles al español. El formulario:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <form action="#" th:action="@{/}" th:object="${persona}" method="post">
        <table>
            <tr>
                <td>Nombre</td>
                <td><input type="text" th:field="*{nombre}" /></td>
                <td th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}">ERROR EN EL NOMBRE</td>
            </tr>

            <tr>
                <td>Edad</td>
                <td><input type="text" th:field="*{edad}" /></td>
                <td th:if="${#fields.hasErrors('edad')}" th:errors="*{edad}">ERROR EN LA EDAD</td>
            </tr>

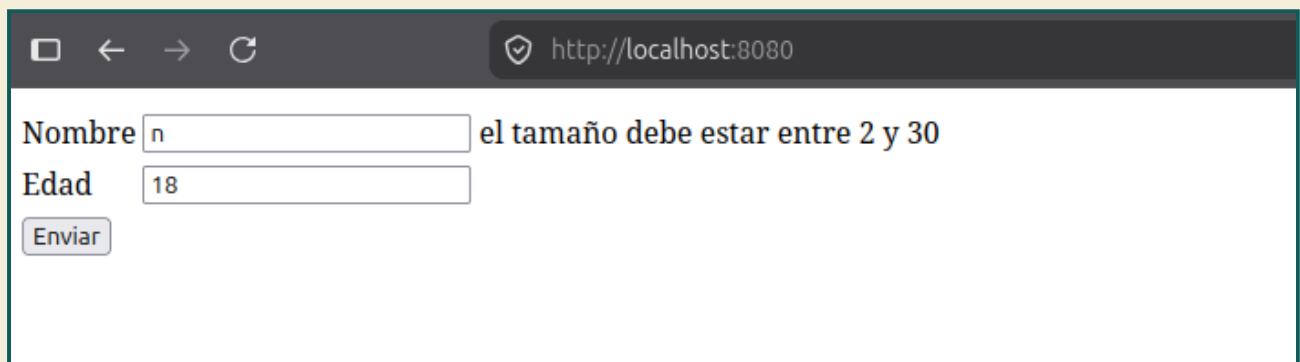
            <tr>
                <td><button type="submit">Enviar</button></td>
            </tr>
        </table>
    </form>
</body>
</html>

```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Resultado</title>
</head>
<body>
  ¡Felicidades! Tue eres lo bastante mayor para estar en este sitio Web
</body>
</html>
```

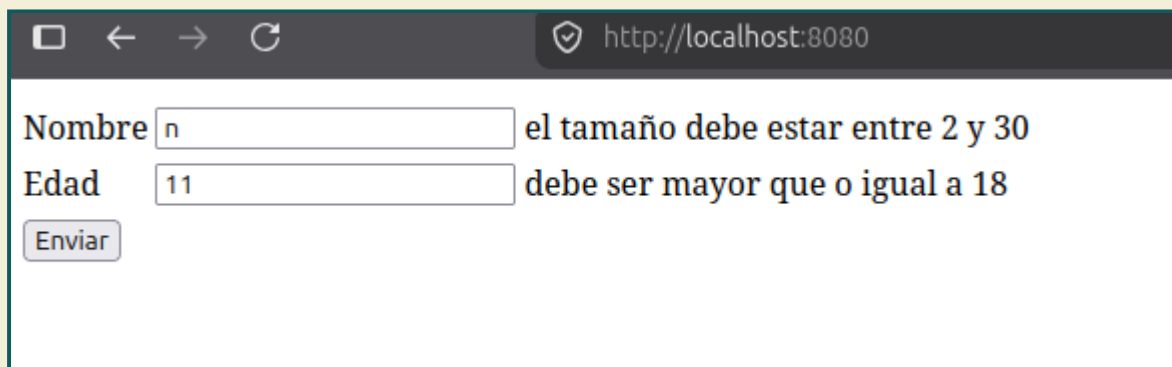
Y ahora adjunto muestras que funciona

Fallo con el nombre



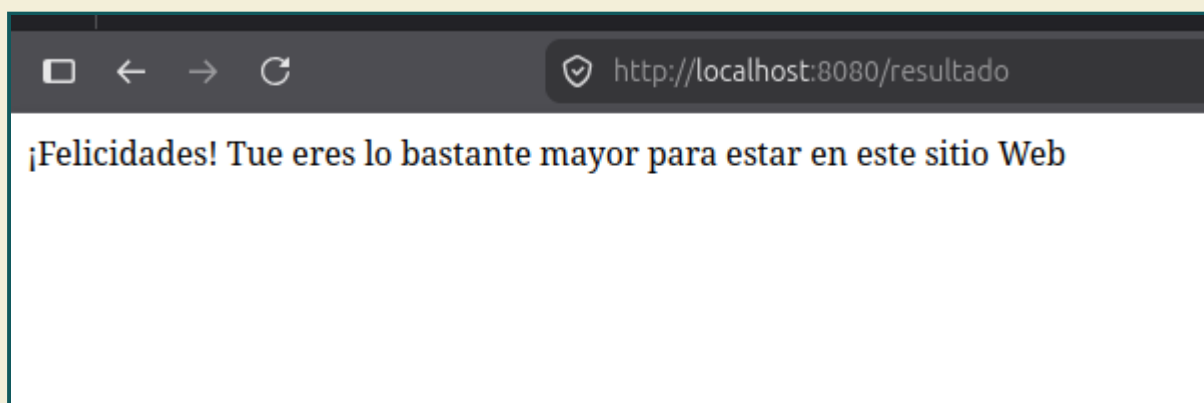
A screenshot of a web browser window at <http://localhost:8080>. The page contains a form with two input fields: "Nombre" and "Edad". The "Nombre" field contains the text "n" and has a red error message next to it: "el tamaño debe estar entre 2 y 30". The "Edad" field contains the number "18". Below the fields is a button labeled "Enviar".

Fallo con la edad



A screenshot of a web browser window at <http://localhost:8080>. The page contains a form with two input fields: "Nombre" and "Edad". The "Nombre" field contains the text "n" and has a red error message next to it: "el tamaño debe estar entre 2 y 30". The "Edad" field contains the number "11" and has a red error message next to it: "debe ser mayor que o igual a 18". Below the fields is a button labeled "Enviar".

Correcto



A screenshot of a web browser window at <http://localhost:8080/resultado>. The page displays the message: "¡Felicidades! Tue eres lo bastante mayor para estar en este sitio Web".

✦ PRACTICA DOS

Aqui teniamos que poner en practica lo aprendido de momento no puedo hacer validaciones por el tiempo pero si las demas

Primero cree un objeto o pojo que me gusta mas utilizarlo:

```
package ruvik.fuerzadelimperio;

import java.util.ArrayList;

public class FormularioImperial {
    private String nombre;

    private Integer edad;

    private ArrayList<String> aficiones;

    private String ramaMilitar;

    private Integer rango;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Integer getEdad() {
        return edad;
    }

    public void setEdad(Integer edad) {
        this.edad = edad;
    }

    public String getRamaMilitar() {
        return ramaMilitar;
    }

    public void setRamaMilitar(String ramaMilitar) {
        this.ramaMilitar = ramaMilitar;
    }

    public Integer getRango() {
        return rango;
    }

    public void setRango(Integer rango) {
        this.rango = rango;
    }

    public ArrayList<String> getAficiones() {
        return aficiones;
    }

    public void setAficiones(ArrayList<String> aficiones) {
        this.aficiones = aficiones;
    }
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
  <h1>FORMULARIO PARA SERVIR AL EMPERADOR</h1>
  <form action="#" th:action="@{/}" th:object="${imperial}" method="post">
    <p>
      Nombre:
      <input type="text" th:field="*{nombre}" />
    </p>

    <p>Edad: <input type="number" th:field="*{edad}" /> </p>
    <p>Aficiones
    </p>
    <input type="checkbox" th:field="*{aficiones}" value="Deporte" /> Deporte
    <input type="checkbox" th:field="*{aficiones}" value="Arte" /> Arte
    <input type="checkbox" th:field="*{aficiones}" value="Fotografia" /> Fotografia
    <select name="ramaMilitar" th:field="*{ramaMilitar}">
      <option value="" disabled selected >-- Selecciona una opción --</option>
      <option value="Astra Militarun">Astra Militarun</option>
      <option value="Ultramarines">Ultramarines</option>
      <option value="Salamandras" >Salamandras</option>
    </select>

    @Controller
    public class ControllerDelImperio {

      @GetMapping("/")
      public String mostrarFormulario(FormularioImperial imperial, Model modelo) {
        modelo.addAttribute("imperial", imperial);

        <p>Rango: <input type="range" th:field="*{rango}" min="1"
                      max="10"
                      step="1"
        /> </p>

        <p><button type="submit">Enviar</button></p>

      </form>
    </body>
  </html>
```

Luego tambien en el controlador donde yo he puesto dos condicones, para vizualizar un tema si eres mayor y de un rango alto:

```
package ruvik.fuerzadelimperio;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ControllerDelImperio {

    @GetMapping("/")
    public String mostrarFormulario(FormularioImperial imperial, Model modelo) {
        modelo.addAttribute("imperial", imperial);

        return "formulario";
    }

    @PostMapping("/")
    public String procesarFormulario(FormularioImperial imperial, Model modelo) {
        modelo.addAttribute("imperial", imperial);

        if(imperial.getEdad()>18) {
            modelo.addAttribute("esMayor",true);
        }
        if(imperial.getRango()>5) {
            modelo.addAttribute("esAltoRango",true);
        }

        return "tucartilla";
    }
}
```

Y el html que debemos devolver es:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
  <h1>BIENVENIDO AL PODEROSO IMPERIO DE LA HUMANIDAD</h1>
  <p>Tu servicio sera en pos del honor, de la gloria. dar tu vida por el emperador</p>

  <h2>Tus datos</h2>

  <p th:text="|Te Llamas: ${imperial.nombre}|"></p>
  <p th:text="|Tu edad es: ${imperial.edad}|"></p>
  <p th:each="aficion : ${imperial.aficiones}" th:text="|aficiones son: ${aficion}|">
  <p th:text="|Petenece la gloria: ${imperial.ramaMilitar}|"></p>
  <p th:text="|Tu Rango es: ${imperial.rango}|"></p>

  <div th:if="${esMayor}">
    <p>Eres un tio que entiende que esto va ser un paseo. Aunque servir al emperador
      pero genial, tu vida aunque no sea recordada seguro seras recordado por algo o
    </p>
  </div>

  <div th:if="${esAltoRango}">
    <p>
      Comisario, Capitan del capitulo, Astarte.
      Hay muchos planetas bajo ataque, actualmente estamos luchando varios frente.
    </p>

    <h1>Planetas</h1>
    <h2>Terra</h2>
    <h2>Arcadia</h2>
  </div>
</body>
</html>
```

Primero siendo menores y de bajo rango

FORMULARIO PARA SERVIR AL EMPERADOR

Nombre:

Edad:

Aficiones

☒ Deporte ☒ Arte ☒ Fotografia

Rango:

y el resutlado es

BIENVENIDO AL PODEROSO IMPERIO DE LA HUMANIDAD

Tu servicio sera en pos del honor, de la gloria. dar tu vida por el emperador

Tus datos

Te Llamas: Ruvik

Tu edad es: 11

aficiones son: Deporte

aficiones son: Arte

aficiones son: Fotografia

Peteneces la gloria: Astra Militarun

Tu Rango es: 1

← → ↺

http://localhost:8080

BIENVENIDO AL PODEROSO IMPERIO DE LA HUMANIDAD

Tu servicio sera en pos del honor, de la gloria. dar tu vida por el emperador

Tus datos

Te Llamas: Ruvik

Tu edad es: 19

aficiones son: Deporte

aficiones son: Arte

aficiones son: Fotografia

Peteneces la gloria: Salamandras

Tu Rango es: 10

Eres un tio que entiende que esto va ser un paseo. Aunque servir al emperador es glorioso. Claramente esto va traer destraste y seguramente mueras en menos de unos minutos pero genial, tu vida aunque no sea recordarda seguro seras recordado por algo o por alguien, ¡ANIMO!

Comisario, Capitan del capitulo, Astarte. Hay muchos planetas bajo ataque, actualmente estamos luchando varios frente. Tu regimiento debe ir aun sitio

Planetas

Terra

Arcadia

✦ CONCLUSIÓN

Muchos conceptos nuevos, validaciones simple se me ha complicado, pero el tema de objeto y modelos me gusta mucho, lo malo son las etiquetas en html que van cambiando segun lo uses.